

# LLM Inference Assignment

Theory, Hardware/Software Exploration, and Coding Exploration

September 3, 2026

## Theory

### 1. Why is inference memory bandwidth-bound important?

When people first think about what makes a GPU fast for LLM inference, the instinct is to look at TFLOPS — the raw compute power. But for the actual token-by-token generation step (the "decode" phase), compute is usually *not* the bottleneck. Memory bandwidth is.

Here's why. During autoregressive decoding, the model generates one token at a time. To generate that single token, the GPU has to load every single weight of the model from its memory (HBM/VRAM) into the compute units, do a relatively small amount of math with it, and then throw it away and do it all again for the next token. The amount of *compute* per token is tiny compared to the amount of *data movement* required to even get the weights to the compute units.

So the GPU spends most of its time waiting to fetch weights from memory rather than doing math. This is what "memory bandwidth-bound" means: the speed of the memory bus (GB/s) — not the number of FLOPS the chip can do — decides how fast tokens come out.

We can even write a rough formula for this. For a dense model with  $P$  parameters, at a given precision (say 2 bytes per parameter for FP16/BF16), each new token requires reading roughly:

$$\text{Bytes read per token} \approx P \times \text{bytes per parameter}$$

And the theoretical maximum tokens/sec is bounded by:

$$\text{Tokens/sec} \leq \frac{\text{Memory Bandwidth (GB/s)}}{\text{Model size (GB)}}$$

For example, a 7B parameter model in FP16 is about 14 GB. On a GPU with 1 TB/s of bandwidth, the theoretical ceiling is roughly  $1000/14 \approx 71$  tokens/sec for a single request (batch size 1), no matter how many TFLOPS the card has on paper.

This is why the industry cares so much about bandwidth-focused numbers like HBM3's 3+ TB/s on the H100, or why techniques like quantization (shrinking the model to 8-bit or 4-bit) help so much — a smaller model means fewer bytes to move per token, which directly translates to faster generation. It's also why batching helps: if you can serve many requests at once, you amortize that same weight-loading cost across many tokens generated in parallel, which shifts the workload from being purely bandwidth-bound toward being more compute-bound and gets you much better GPU utilization.

In short: for LLM inference, especially with small batch sizes and long generations, the question "how fast can I move data off the chip's memory" matters more than "how many operations can

the chip do per second.” This is the single biggest reason inference optimization work (KV-cache management, quantization, continuous batching, speculative decoding) exists — almost all of it is really about reducing memory traffic or making better use of the bandwidth that’s available.

## 2. How can we calculate the overall memory required to store a model? How are LLM models represented while storing?

### How models are represented while stored

An LLM is essentially a huge collection of numbers — the weights (and biases) of every linear layer, attention head, embedding table, and normalization layer in the network. When we store a model, we’re storing these numbers in some numeric format, and the format we pick determines how many bytes each number takes up:

- **FP32 (32-bit float)** — 4 bytes/parameter. Used in older training setups, rarely used for storing modern LLMs because it wastes space.
- **FP16 / BF16 (16-bit float)** — 2 bytes/parameter. This is the standard ”full precision” format most LLMs are released in today.
- **INT8 / FP8 (8-bit)** — 1 byte/parameter. A quantized format, roughly half the size of FP16 with usually a small accuracy trade-off.
- **INT4 / 4-bit formats (e.g., GPTQ, AWQ, GGUF Q4)** — 0.5 bytes/parameter. Very compact, popular for running large models on consumer GPUs.

On disk, these weights are usually saved in a container format like **safetensors** (the modern standard — safe, fast to memory-map, and doesn’t allow arbitrary code execution the way old **pickle**-based **.bin** checkpoints did), alongside a config file (**config.json**) describing the architecture (number of layers, hidden size, number of attention heads, vocabulary size, etc.) and a tokenizer file. So ”the model” on disk is really: architecture metadata + a big folder of tensors, sharded across multiple files if it’s large.

### Calculating overall memory required

The most useful back-of-envelope formula is:

$$\text{Model size (bytes)} = \text{Number of parameters} \times \text{Bytes per parameter}$$

So for a model with  $P$  parameters at precision using  $b$  bytes:

$$\text{Size (GB)} = \frac{P \times b}{1024^3}$$

Example: LLaMA-2 7B has about 7 billion parameters.

- At FP16 (2 bytes):  $7\text{B} \times 2 = 14\text{ GB}$
- At INT8 (1 byte):  $7\text{B} \times 1 = 7\text{ GB}$
- At INT4 (0.5 bytes):  $7\text{B} \times 0.5 = 3.5\text{ GB}$

But that’s only the weights. When we actually want to *run* the model (i.e., calculate the total VRAM needed for inference, not just storage), we need to add a few more things:

1. **Model weights** — as calculated above.

2. **KV cache** — During generation, the model caches the Key and Value tensors for every previous token so it doesn't have to recompute attention from scratch each step. This grows with sequence length, batch size, number of layers, and number of attention heads, and it can easily become the biggest memory consumer for long-context or high-concurrency serving. Roughly:

$$\text{KV cache size} \approx 2 \times \text{layers} \times \text{hidden size} \times \text{seq length} \times \text{batch size} \times \text{bytes per value}$$

(the factor of 2 is because we store both Keys and Values.)

3. **Activations / runtime overhead** — intermediate tensors computed during the forward pass, plus framework and CUDA context overhead. This is usually much smaller than the KV cache during inference but still needs to be budgeted for, typically as a percentage buffer (people often just add 10–20% headroom on top of weights + KV cache as a rule of thumb).

So the full estimate looks like:

$$\text{Total VRAM needed} = \text{Weights} + \text{KV Cache} + \text{Activation/overhead buffer}$$

This is exactly why a 7B model "needs 14 GB" is a simplification — in production, with a long context window and many concurrent users, the KV cache can dwarf the weight size, which is the whole reason techniques like PagedAttention (used in vLLM) and quantized KV caches exist.

### 3. Explain each layer of inferencing and its importance to an institution/-company

I like to think about "the inference stack" as a set of layers stacked on top of each other, each solving a different problem. Here's how I'd break it down, from the bottom (physical hardware) to the top (the actual application the business cares about):

1. **Hardware layer (GPUs/accelerators + memory + interconnect).** This is the physical foundation — GPUs (or TPUs, custom ASICs), their VRAM, and how they're connected (NVLink, PCIe, InfiniBand for multi-node setups). For a company, this layer decides the ceiling on how large a model you can even load, and how many requests per second you can physically push through. Getting this wrong (e.g., under-provisioning VRAM or bandwidth) means every layer above it inherits that bottleneck — no amount of clever software fixes a GPU that simply doesn't have enough memory bandwidth for your traffic.
2. **Model/runtime layer (the model weights + execution engine, e.g., PyTorch, ONNX Runtime, TensorRT).** This layer is responsible for actually executing the neural network's forward pass efficiently on the hardware — using optimized kernels (like FlashAttention), fusing operations, and picking the right numeric precision (FP16/INT8/INT4). For a business, this is where a lot of the cost savings come from: a well-optimized runtime can get 2–4x more throughput out of the exact same GPU, which directly translates to lower cost per request.
3. **Serving layer (the inference server, e.g., vLLM, TGI, TensorRT-LLM, SGLang).** This is the layer that turns a raw model execution engine into something that can actually serve many users at once. It handles continuous/dynamic batching (grouping requests together so the GPU stays busy), KV-cache management, request scheduling, and exposing an API (usually OpenAI-compatible). For an institution, this layer is the difference

between "a model that works in a demo" and "a model that can serve thousands of concurrent customers without falling over." This is usually where most of the engineering effort in a production LLM team goes.

4. **Orchestration/infrastructure layer (Kubernetes, autoscalers, load balancers, multi-node schedulers).** Once you have one serving instance working, a real company needs many of them, running across many machines, scaling up and down with demand, with failover if a node dies. This layer is what makes the system reliable and cost-efficient at scale — auto-scaling down at 3 AM saves real money, and failover keeps the product up during hardware failures.
5. **API / application layer.** This is what the end user or downstream application actually talks to — a REST/gRPC API, rate limiting, authentication, logging, request routing, caching of repeated queries, guardrails/content filtering, and so on. For a business this layer is where product requirements (latency SLAs, safety policies, cost controls, multi-tenant isolation) get enforced. It's also where you'd plug in things like prompt caching or semantic caching to cut costs further.
6. **Observability governance layer.** Monitoring latency (TTFT — time to first token, and inter-token latency), throughput, GPU utilization, cost per request, and quality metrics; plus logging for compliance and auditing. For a company operating at any real scale, this layer is what lets you know *why* something is slow or expensive, and it's essential for meeting SLAs and regulatory requirements (especially in regulated industries like finance or healthcare).

Why does this layered view matter for an institution? Because each layer represents a different lever for cost, performance, and reliability, and different teams typically own different layers. A company deciding "should we build our own inference stack or use a managed API" is really asking: how much of this stack do we want to own and optimize ourselves, versus pay someone else to handle? Understanding each layer lets a business make informed trade-offs — e.g., a startup might rent a managed API and only worry about the top two layers, while a large company running LLMs at massive scale (to control cost per token) will invest heavily in the hardware, runtime, and serving layers because even small efficiency gains multiply into huge savings when you're serving millions of requests a day.

## Hardware & Software Exploration

### 4. Pick any 2 GPUs, find VRAM, bandwidth, and rental cost. Calculate the biggest model each can hold at 16-bit and 4-bit.

I picked the **NVIDIA H100 (SXM)** as a high-end data-center GPU, and the **NVIDIA RTX 4090** as a consumer-grade GPU, since they sit at very different points on the price/performance spectrum and it's a useful contrast.

| Spec                     |       | NVIDIA H100 SXM                                  | NVIDIA RTX 4090                        |
|--------------------------|-------|--------------------------------------------------|----------------------------------------|
| VRAM                     |       | 80 GB HBM3                                       | 24 GB GDDR6X                           |
| Memory bandwidth         | band- | ≈3.35 TB/s                                       | ≈1.0 TB/s (1,008 GB/s)                 |
| Typical on-demand rental | on-   | ≈\$1.99–\$2.99/hr<br>(community/on-demand cloud) | ≈\$0.34–\$0.45/hr<br>(on-demand cloud) |

**Method:** I'll use the simple rule that VRAM available for weights is roughly 90% of total VRAM (leaving room for the KV cache and runtime overhead), and that a model needs 2 bytes/parameter at 16-bit and 0.5 bytes/parameter at 4-bit.

$$\text{Max params} = \frac{\text{Usable VRAM (GB)} \times 1024^3}{\text{bytes per parameter}}$$

#### H100 SXM (80 GB, usable ≈ 72 GB):

- At 16-bit (2 bytes/param):  $\frac{72 \text{ GB}}{2} \approx 36$  billion parameters → comfortably fits a 34B model with headroom for a solid KV cache; roughly the ceiling for a dense ~36B model.
- At 4-bit (0.5 bytes/param):  $\frac{72 \text{ GB}}{0.5} \approx 144$  billion parameters → can hold something in the 70B–120B range comfortably (e.g., a quantized Llama-3 70B fits with plenty of room to spare).

#### RTX 4090 (24 GB, usable ≈ 21.6 GB):

- At 16-bit (2 bytes/param):  $\frac{21.6 \text{ GB}}{2} \approx 10.8$  billion parameters → fits a ~7B–8B model comfortably (e.g., Llama-3 8B, Mistral 7B) with room for a modest KV cache; a 13B model would be tight or need reduced context.
- At 4-bit (0.5 bytes/param):  $\frac{21.6 \text{ GB}}{0.5} \approx 43$  billion parameters → can run a heavily quantized ~30B–34B model (e.g., a 4-bit GGUF/AWQ build of a 32B model).

**Takeaway:** the H100 costs roughly 5–8x more per hour than the RTX 4090, but it gives you over 3x the memory bandwidth and more than 3x the usable VRAM — so for a single large model, the H100 is the only realistic option, whereas for smaller (7B–13B class) models the RTX 4090 is a much cheaper way to get decent throughput, especially once quantized.

### 5. Research 2 serving frameworks (TGI, Ollama, LMDeploy, etc.) not covered in depth

I chose to look into **Hugging Face TGI (Text Generation Inference)** and **Ollama**, since they represent two very different philosophies — one built for production serving, the other

built for local simplicity.

## **Hugging Face TGI**

TGI is a Rust/Python/gRPC server built by Hugging Face, originally created to power products like HuggingChat and HF’s Inference Endpoints. It supports continuous batching, tensor parallelism across multiple GPUs, Flash Attention/Paged Attention kernels, quantization (bit-sandbytes, GPT-Q), and token streaming, and it’s tightly integrated with the Hugging Face ecosystem (model hub, safetensors, tokenizers) — meaning if your models and workflow already live on the HF Hub, TGI is a very natural fit with minimal glue code. One important caveat from my research: as of 2026, Hugging Face has effectively put TGI into maintenance mode and is steering new users toward vLLM and SGLang for new projects, so it’s still solid for existing, stable deployments but isn’t where active feature development is happening anymore.

*When I’d pick it over vLLM/SGLang/TensorRT-LLM:* if a team is already deeply embedded in the Hugging Face ecosystem (models, tokenizers, existing HF-based CI/CD) and just needs a reliable, well-documented server without wanting to adopt a newer, faster-moving project — or if they’re maintaining an existing TGI deployment where migration cost isn’t worth it yet.

## **Ollama**

Ollama is a local-first inference tool built on top of llama.cpp, designed to make running an LLM on your own laptop as easy as a single command (`ollama run llama3.2`). It handles model download, quantized GGUF weight management, and exposes a simple local API, all with almost zero configuration. The trade-off is that it’s built for single-user, single-request workloads — it doesn’t do continuous batching or serve many concurrent requests efficiently the way production servers do, so it’s not meant to be a scalable backend.

*When I’d pick it over vLLM/SGLang/TensorRT-LLM:* for local development, quick prototyping, personal projects, or small internal tools where only one or a handful of people are using the model at a time and ease of setup matters far more than raw throughput. I would never pick Ollama for a customer-facing production endpoint expecting real concurrency — that’s exactly the situation vLLM, SGLang, or TensorRT-LLM are built for.

## Coding Exploration

### 6. Run `nvidia-smi` on any GPU machine and explain at least 5 things in the output

Below is a representative `nvidia-smi` output (from a machine with an NVIDIA GPU), followed by an explanation of what the key fields mean.

|                                                                   |  |  |  |  |  |  |  |  |  |
|-------------------------------------------------------------------|--|--|--|--|--|--|--|--|--|
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |
| NVIDIA-SMI 550.90.07 Driver Version: 550.90.07 CUDA Version: 12.4 |  |  |  |  |  |  |  |  |  |
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |
| GPU Name Persistence-M   Bus-Id Disp.A   Volatile Uncorr. ECC     |  |  |  |  |  |  |  |  |  |
| Fan Temp Perf Pwr:Usage/Cap  Memory-Usage   GPU-Util Compute M.   |  |  |  |  |  |  |  |  |  |
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |
| 0 NVIDIA H100 80GB HBM3 On   00000000:00:04.0 Off   0             |  |  |  |  |  |  |  |  |  |
| N/A 34C P0 72W / 700W   14235MiB / 81559MiB   23% Default         |  |  |  |  |  |  |  |  |  |
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |
| Processes:                                                        |  |  |  |  |  |  |  |  |  |
| GPU GI CI PID Type Process name GPU Memory                        |  |  |  |  |  |  |  |  |  |
| ID ID Usage                                                       |  |  |  |  |  |  |  |  |  |
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |
| 0 N/A N/A 18342 C python3 (vllm serve process) 14180MiB           |  |  |  |  |  |  |  |  |  |
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+     |  |  |  |  |  |  |  |  |  |

Here are at least five things this output tells us, and why they matter:

1. **Driver Version & CUDA Version** (top line) — this tells us which NVIDIA driver is installed (550.90.07) and the maximum CUDA toolkit version that driver supports (12.4). This matters a lot in practice: if a framework (like PyTorch or vLLM) needs a newer CUDA version than the installed driver supports, you'll get cryptic compatibility errors, so this is usually the first thing to check when debugging "why won't my GPU code run."
2. **GPU Name** (NVIDIA H100 80GB HBM3) — identifies exactly which GPU model is installed, which tells you the compute capability, VRAM, and expected performance profile you're working with — important when you're trying to figure out if a model will even fit.
3. **Memory-Usage** (14235MiB / 81559MiB) — shows how much VRAM is currently in use versus the total available. This is the number you watch to know if you're close to an out-of-memory (OOM) crash, and it's the first thing I'd check before trying to load another model or increase batch size.
4. **GPU-Util** (23%) — the percentage of time, over the last sampling period, that at least one GPU kernel was running. Low utilization while a process is "running" is often a smoking gun for a bottleneck elsewhere (e.g., CPU preprocessing, slow data loading, or being memory-bandwidth bound rather than compute bound), so this metric is central to performance debugging.

5. **Temp & Power (Pwr:Usage/Cap)** (34C, 72W / 700W) — the current GPU die temperature and current power draw versus its power cap (700W for this H100). If temperature climbs too high, the GPU will automatically throttle its clock speed to protect itself, silently reducing performance — so this is a useful early warning sign when performance mysteriously drops under sustained load.
6. **Processes table (PID, Process name, GPU Memory Usage)** — lists exactly which processes are using the GPU and how much memory each one is consuming. This is essential when multiple people or jobs share a machine — it tells you who/what to blame if the GPU is full, and it's the fastest way to find and kill a stuck or leaked process holding onto memory.
7. **Persistence Mode (Persistence-M)** — shows whether the driver stays loaded in memory even when no application is using the GPU. Keeping this "On" avoids the extra latency of reinitializing the driver on every new job, which matters for services that need to spin up GPU processes quickly and repeatedly.

As a quick note on the "how do GPU commands actually run" prerequisite: `nvidia-smi` itself doesn't run *on* the GPU — it's a CPU-side management utility that talks to the NVIDIA driver via a low-level management library (NVML) to query the GPU's state. The actual model computation happens through a completely separate path: a framework like PyTorch calls into CUDA, which compiles and launches kernels that run directly on the GPU's streaming multiprocessors, using the GPU's own memory (VRAM) rather than system RAM. `nvidia-smi` is just a window into what the driver already knows about that separate, ongoing computation.